

Arepo: Prediction-Market Intelligence Platform

An independently developed, read-only prediction-market intelligence platform using live Polymarket public data, event-driven ingestion, order-book microstructure analysis, transparent anomaly detection, and deterministic historical replay.

Disclaimer. Arepo is a research and engineering tool. It is not a betting product and never places trades. It flags statistically unusual market behaviour for further investigation: this is not evidence of insider activity, and the platform makes no claim of profitability or validated predictive alpha.

1. Executive summary

Arepo turns the raw, public data of a prediction-market venue into an explainable, quantitative view of how each market is behaving *right now* and how unusual that behaviour is relative to the market's own recent history. It ingests market discovery metadata and order-book data from Polymarket's public Gamma and CLOB interfaces, normalises it through a strict typed boundary, computes a small set of transparent microstructure and time-series statistics, and serves them through a FastAPI backend to a Next.js interface.

The system is deliberately narrow and correct rather than broad and shallow. Every analytic is documented with its formula, edge-case behaviour, and limitations; every response carries a provenance envelope stating whether the data is **live**, **cached**, or **replay**, so the interface can never silently present stale or synthetic data as live. A deterministic, version-controlled replay dataset drives a look-ahead-safe backtest, making the anomaly signal reproducible and inspectable.

Status at the time of writing: the backend is complete and verified: 121 automated tests pass and static analysis is clean. Live discovery and order-book analysis have been exercised against the real Polymarket APIs; the cached path has been verified with a real ingestion round-trip; and the deterministic replay/backtest produces stable, non-trivial results. Docker and public-deployment configurations are provided and validated by inspection but were not executed in the build environment (see §8).

2. Problem and motivation

A price on a binary prediction-market contract is often read directly as "the probability of the event." That reading is convenient but incomplete: the price is a risk-neutral, spread-and-fee-contaminated estimate, and a single number hides everything about *how* the market arrived there: how deep the book is, how wide the spread is, whether buying pressure is lopsided, and whether the recent move is ordinary or extreme for that market.

Arepo's motivation is to make that microstructure legible. Rather than predicting outcomes, it answers a narrower and more defensible question: **given a market's own recent history, how unusual is its current behaviour, and how much should we trust that reading?** This is the kind of screening tool a small quantitative desk builds internally to decide *where to look*, not *what to bet*. Framing the problem as explainable anomaly screening: with explicit data-quality and confidence: keeps the project honest and technically substantial without overclaiming.

3. Data sources

All data comes from Polymarket's **public, read-only** interfaces. No authentication, wallet, or private key is used, and no trading endpoint is ever called. Endpoint shapes were verified by direct requests during development (see docs/research-notes.md).

Interface	Base	Used for
Gamma API	gamma-api.polymarket.com	Market/event discovery, metadata, tags, outcome prices, CLOB token ids
CLOB REST	clob.polymarket.com	Order book, midpoint, price, spread, price history
CLOB market WebSocket	wss://ws-subscriptions-clob.polymarket.com/ws/market	Live book snapshots, price changes, trades (event-driven)

Two properties of the upstream data shaped the design. First, structured fields arrive as **JSON-encoded strings** (e.g. `outcomes`, `outcomePrices`, `clobTokenIds`) and numbers arrive inconsistently as strings and/or `...Num` variants: so a defensive normalisation layer is essential. Second, documentation and live behaviour can diverge (the `/midpoint` endpoint is documented to return `mid_price` but was observed returning `mid`); the client accepts both, and direct probes are treated as ground truth where they disagree with docs.

4. Architecture

Arepo is a monorepo with a Python backend and a TypeScript frontend, organised by clear module boundaries:

- **domain**: typed Pydantic models that form the internal contract (`Market`, `OrderBook`, `Signal`, `DataStatus`, ...). All timestamps are timezone-aware UTC.
- **clients**: async httpx clients for Gamma and CLOB REST, and a resilient websockets client for the CLOB market channel (bounded exponential reconnect, resubscribe, LRU de-duplication by message hash, PING/PONG heartbeat, stale detection, and graceful signalling to fall back to REST polling).
- **ingest**: an anti-corruption layer that normalises raw upstream JSON into domain models, plus a pipeline that discovers markets and snapshots order books into storage.
- **storage**: async SQLAlchemy (SQLite locally, Postgres-compatible) for a market cache, a snapshot time-series, and per-source health.
- **analytics**: the quantitative core (§5): pure, NumPy-based functions on plain numeric series, so they are trivially testable and identical across data modes.
- **replay**: a deterministic, version-controlled dataset and a player that serves it through the same domain models, plus a look-ahead-safe backtest.
- **service**: the application "brain": it selects a data source (live/cached/replay) with graceful fallback, enriches markets with the shared analytics, and assembles API responses.
- **api**: a FastAPI layer with structured logging, request timing, a safe error handler that never leaks stack traces, and an interactive OpenAPI page.
- **frontend**: a Next.js/TypeScript/Tailwind interface with an always-visible data-mode and health indicator.

A central design rule is that **raw third-party JSON never reaches the frontend**: it is validated and normalised once, at the ingestion boundary, and only typed internal models are serialised outward. A second rule is that **the data mode travels with the data**: every response carries a `DataStatus` envelope (mode, REST/WebSocket health, last update, data age, degradation reason), so cached or replay data can never be mistaken for live. Architecture, sequence, and fallback diagrams are in `docs/architecture.md`.

5. Quantitative methodology

The analytics are intentionally few, transparent, and correct. Full formulas and edge-case handling are in `docs/methodology.md`; the essentials:

- **Implied probability**. A binary contract price is treated as an approximate, risk-neutral implied probability, clamped to $[0, 1]$, with the mid-price preferred over a one-sided trade price. Across a market's outcomes, a normalised view divides by the sum to remove the "overround." The caveats are surfaced, not hidden.
- **Returns and movement**. For bounded probability series, returns default to **additive first differences** (probability-point changes) rather than relative/log returns, which explode near the 0/1 boundaries. Movement is reported as percentage-point and relative change over configurable windows.
- **Rolling volatility**. Sample standard deviation ($\text{ddof} = 1$) of returns over a defined window, with an explicit insufficient-history guard.
- **Standardised movement (z-score)**. $z = (r - \text{mean}) / \text{std}$ over a rolling window, with explicit handling of zero variance (no meaningful standardised move), insufficient history, missing values, and optional winsorisation/clipping of outliers.
- **Microstructure**. Midpoint, absolute and relative spread, spread-in-ticks; order-book imbalance $(\text{bid_depth} - \text{ask_depth}) / (\text{bid_depth} + \text{ask_depth})$ over the first N levels; and a transparently-defined *near-mid executable depth* within a stated band of the midpoint. Every measure returns `None` rather than raising on empty or one-sided books.
- **Composite anomaly score**. A weighted mean of normalised, individually-visible components ($|\text{return z-score}|$, volume acceleration, book imbalance, spread change, depth change). Each component saturates at a documented cap; weights renormalise over the components actually available, so a missing input neither inflates nor deflates the score. The default weights are exposed as *assumptions*, not claimed to be empirically optimal.

- **Confidence and data quality.** A multiplicative penalty model reduces confidence for short history, wide spread, thin book, stale data, one-sided books, and partial API coverage: and records a human-readable reason for every penalty, so a low score is always explainable. A signal's reported confidence is its strength multiplied by this data-quality factor.
- **Look-ahead-safe backtest.** Over the deterministic replay dataset, a signal at frame i is computed using only frames $0 \dots i$; its outcome is evaluated using only frames $i+1 \dots i+H$. Signal-generation and evaluation data never overlap (this is asserted in the loop). The backtest reports thresholds, horizon, sample size, missing observations, a directional follow-through "hit rate," a false-positive rate, and average forward movement: with explicit assumptions (no transaction costs modelled) and limitations (synthetic data, small sample, no survivorship correction). It measures follow-through, **not** profitability.

6. Testing and validation

Correctness is enforced by an automated suite (**121 tests, all passing; linting clean**) that covers, among other cases: implied probability and normalisation; movement, rolling volatility, and z-score (including zero-variance and insufficient-history edge cases, with several **hand-derived** expected values); midpoint/spread/imbalance including zero-denominator and empty/one-sided books; normalisation of malformed and missing upstream fields; order-book best-first sorting; duplicate-event handling and reconnection logic in the WebSocket client; the live→cached→replay fallback; storage round-trips; the ingestion pipeline; the API surface; and the backtest's determinism and look-ahead-safety (the "prefix" property is tested directly).

Beyond unit tests, each backend subsystem was implemented independently and then reviewed by a separate agent and personally verified by running the tests and inspecting the code and its behaviour against the real APIs. This process caught at least one real defect (a client that leaked a raw HTTP-library error on a 404, which would have crashed a market-detail request for resolved markets); it was fixed by mapping all non-retryable client errors into a typed hierarchy and adding regression tests.

7. Example results

Real observations (live mode). Against the live Polymarket APIs, discovery returns real, active markets across categories (e.g. Politics, Crypto, Sports). For a representative market, the platform pulled a real two-sided order book, computed an implied probability from the midpoint, a spread of ~0.02, a strongly one-sided order-book imbalance, and a z-score computed over ~145 real price-history points: surfacing a high composite-anomaly reading driven by the extreme imbalance and recent move. These are illustrative single-market observations, not a statistical claim about the venue.

Replay results (deterministic). On the committed synthetic dataset (three markets, 144 frames, with planted momentum and spike-and-revert episodes), the backtest is fully reproducible and produces a non-degenerate result: **16 signals evaluated, a directional follow-through hit rate of 0.625, and a false-positive rate of 0.125**. By construction, the momentum episodes follow through (hits) and the spike-and-revert episodes do not (misses). This demonstrates the end-to-end signal→evaluation machinery and its look-ahead safety; because the dataset is synthetic and small, the numbers are illustrative and **do not generalise to live markets or imply profitability**.

8. Limitations

The honest limitations are documented in full in `docs/limitations.md`. The most important:

- Implied probabilities are spread/fee-contaminated risk-neutral estimates, not calibrated forecasts.
- Signals are **screening heuristics**. They are not evidence of insider trading, not a profitability signal, and not validated predictive alpha.
- The backtest runs on a **synthetic, deterministic** dataset; its results are illustrative only. There is no survivorship correction and no transaction-cost model.
- The anomaly weights, saturation caps, and windows are assumptions a production desk would re-derive from labelled data.
- Per-token historical *volume* is not readily available from the public CLOB, so the volume-acceleration component uses limited inputs in live mode.
- **Docker startup and public deployment were not executed in the build environment** (no Docker daemon and no deployment CLI/host authentication were available). The container and deployment configurations are provided and validated by inspection; local execution was verified via the non-Docker path (uvicorn + Next.js dev server).

9. Lessons

- **Primary sources beat tutorials.** Verifying endpoint shapes by direct request caught a live/documentation divergence that would otherwise have produced subtle bugs.
- **A typed anti-corruption boundary pays for itself.** Absorbing all upstream messiness in one place kept the analytics, storage, and API clean and made defensive behaviour testable.
- **Make provenance a first-class value.** Carrying the data mode with every response removed an entire class of "is this live?" ambiguity and made the reliability modes trivial to surface.
- **Independent review and personal verification find real bugs.** The 404-handling defect was invisible to the happy-path tests and only surfaced when the system was driven against real, imperfect data.

10. Future improvements

- Persist the live WebSocket feed into the snapshot store to build a genuine live price history and enable richer volume-acceleration and depth-change components.
- Empirically derive and validate the composite-anomaly weights against labelled historical episodes, with calibration and reliability diagnostics.
- Extend the backtest to real recorded data with survivorship handling and a simple transaction-cost/spread model, reported strictly as follow-through statistics.
- Add multi-outcome (non-binary) analytics and category-level baselines.
- Execute the Docker stack and a public deployment, then add production smoke tests and a verified live-demo URL.

Prepared as an independent engineering and research portfolio project. All functionality described has been implemented and verified as stated; no accuracy, profitability, or deployment result is claimed beyond what is written here.